Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

STRATOS

Signal Time-domain Research and Analysis Tools for ObservationS

Pipeline Documentation | Version 1.0

Single-pulse detection • Multi-Gaussian fitting • Fluence measurement

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

Table of Contents

Contents

1 Overview . . . 3

2 Installation . . . 4
2.1 Requirements . . . 4
2.2 Install via pip . . . 4
2.3 Verify Installation . . . 4

3 Quick Start . . . 5

4 Command Reference . . . 6
4.1 `read` — Load and dedisperse a filterbank file . . . 6
4.2 `clean` — RFI masking and timeseries extraction . . . 7
4.3 `extract` — Individual pulse detection . . . 8
4.4 `fit` — Iterative multi-Gaussian decomposition . . . 9
4.5 `fluence` — Radiometric fluence & width measurement . . . 10
4.6 `energy` — Convert fluence to spectral energy . . . 11
4.7 `run` — Execute the full pipeline end-to-end . . . 12

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

1. Overview

STRATOS is a modular, command-line pulsar single-pulse analysis pipeline written in Python. It processes raw filterbank data from radio telescopes, detects individual pulses, characterises their morphology through iterative multi-Gaussian decomposition, and measures their radiometric properties.

The pipeline is structured as a chain of discrete, independently callable steps:

Each step can be invoked individually, making it easy to re-run only the parts of the pipeline that need changing. The `run` command chains all steps automatically for a complete end-to-end workflow.

Key Features

- Reads standard SIGPROC filterbank (`.fil`) files via **sigpyproc**
- Flexible RFI channel masking with arbitrary channel-range pairs
- Automatic peak detection with configurable prominence and separation
- Iterative multi-Gaussian fitting driven by reduced χ^2 minimisation
- FWHM-based pulse envelope definition — no manual width entry required
- Radiometer-equation fluence and energy measurements
- Diagnostic PDF output with per-pulse fit plots
- Fully scriptable CLI suitable for batch processing thousands of pulses

© 2025 All rights reserved

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

2. Installation

2.1 Requirements

Package	Version	Purpose
Python	≥ 3.8	Runtime
numpy	≥ 1.22	Array mathematics
scipy	≥ 1.8	Curve fitting, peak finding
matplotlib	≥ 3.5	Diagnostic plots
sigpyproc	≥ 0.9	Filterbank I/O
astropy	≥ 5.0	Time / coordinate utilities

2.2 Install via pip

```
# Create and activate a virtual environment (recommended)
python -m venv stratos-env
source stratos-env/bin/activate        # Linux / macOS
stratos-env\Scripts\activate           # Windows

# Install dependencies
pip install numpy scipy matplotlib astropy sigpyproc

# Clone the repository
git clone https://github.com/your-org/stratos.git
cd stratos
```

2.3 Verify Installation

```
python stratos.py --help
```

Note

sigpyproc is only required for the `read` command and the full `run` pipeline. All other steps operate on plain text files and do not require sigpyproc.

© 2025 All rights reserved

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

3. Quick Start

The fastest way to process a new observation is to run the complete pipeline with a single command:

```
python stratos.py run \
    --fil         /data/obs/B0355+54_2024.fil \
    --dm          57.14 \
    --block       4000000 \
    --bad-channels 0 30 490 512 \
    --samp-rate   0.025 \
    --prominence  7 \
    --half-width  250 \
    --chi2-threshold 1.5 \
    --sefd        17 \
    --bw          400e6 \
    --output-dir  ./B0355_results
```

This creates the following output directory tree:

```
B0355_results/
--- timeseries_raw.txt          # dedispersed 2-D block
--- timeseries_cleaned.txt      # RFI-masked, normalised 1-D
    timeseries
--- pulses/
---   --- pulse_P0001.txt         # individual pulse windows
---   --- pulse_P0001_fit.txt     # Gaussian fit results
---   --- pulse_P0002.txt
---   --- pulse_P0002_fit.txt
---   --- ...
--- fit_diagnostics.pdf         # one plot per pulse
--- pulse_properties.txt        # widths + fluences (main output)
```

• Tip

After your first run you can re-run only the fitting step with different thresholds without re-reading the filterbank:

```
python stratos.py fit --pulse-dir ./B0355_results/pulses
--chi2-threshold 2.0
```


© 2025 All rights reserved

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

4. Command Reference

STRATOS exposes seven subcommands. Run `python stratos.py <command> -help` at any time to see the full argument list for a specific command.

4.1 `read` — Load and dedisperse a filterbank file

```
python stratos.py read --fil <file.fil> --dm <DM> --block <N> [-o
    output.txt]
```

Reads a SIGPROC filterbank file, dedisperses the data using the provided dispersion measure, and saves the raw two-dimensional block (channels × samples) to a text file for downstream processing.

Flag	Default	Description
`--fil`	*required*	Path to the input `.fil` filterbank file
`--dm`	*required*	Dispersion measure in $\mathrm{pc\,cm^{-3}}$
`--block`	*required*	Number of time samples to read
`-o / --output`	`timeseries_raw.txt`	Output filename for the 2-D data block

Note

The block size should encompass the entire observation or the segment of interest. Setting it too small will truncate the data; too large may exhaust system memory for very wide-band or long observations.

Example

```
python stratos.py read \
    --fil   /data/obs/B0355_20240310.fil \
    --dm    57.14 \
    --block 8000000 \
    -o      B0355_raw.txt
```


© 2025 All rights reserved

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

4.2 `clean` — RFI masking and timeseries extraction

```
python stratos.py clean \
    --ts <raw.txt> \
    [--bad-channels c1_start c1_end c2_start c2_end ...] \
    [--samp-rate 0.025] \
    [-o cleaned.txt]
```

Masks specified channel ranges by replacing their values with NaN, then collapses the data in frequency (`nanmean`) to produce a one-dimensional S/N timeseries. The timeseries is normalised by subtracting the baseline mean and dividing by the baseline standard deviation (estimated from the first 10% of the data).

Flag	Default	Description
`--ts`	*required*	Path to 2-D raw timeseries from read step
`--bad-channels`	*(none)*	Space-separated pairs: start end start end .
`--samp-rate`	`0.025`	Time resolution in ms per sample
`-o / --output`	`timeseries_cleaned.txt`	Output filename

Bad Channel Syntax

Channels are given as a flat list of pairs. Each pair defines an inclusive start–end range (0-indexed, end exclusive). Multiple ranges are supported:

```
# Mask channels 0-29, 100-149, and 490-511
python stratos.py clean \
    --ts raw.txt \
    --bad-channels 0 30 100 150 490 512 \
    -o cleaned.txt
```

! Warning

Channel indices are 0-based and the end index is **exclusive** (Python-style slicing). For a 512-channel backend, the last valid channel index is 511, so use `490 512` to mask channels 490–511.

© 2025 All rights reserved

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

4.3 `extract` — Individual pulse detection

```
python stratos.py extract \
    --ts <cleaned.txt> \
    [--distance 160] [--prominence 8] [--half-width 200] \
    [--save-dir ./pulses] [--prefix pulse]
```

Detects peaks in the cleaned S/N timeseries using `scipy.signal.find_peaks` and saves a window of ±`half-width` samples around each detected peak as an individual text file. Each file contains two columns: sample index and S/N.

Flag	Default	Description
`--ts`	*required*	Cleaned, normalised timeseries file
`--distance`	`160`	Minimum sample separation between detected peaks
`--prominence`	`8`	Required peak prominence in S/N units
`--half-width`	`200`	Samples to take on either side of each peak
`--save-dir`	`./pulses`	Directory to write individual pulse files
`--prefix`	`pulse`	Filename prefix for output files

Tuning Tips

- **distance**: set to at least one pulse period in samples to avoid double-detecting the same pulse. For B0355+54 ($P \approx 156$ ms, $\delta t = 0.025$ ms) use `distance`≈ 6240.
- **prominence**: increase if noise spikes are being saved as pulses; decrease if faint pulses are being missed.
- **half-width**: should be wide enough to include the full pulse and a noise baseline on each side — at least 3–5× the expected pulse width.

Note

A `<prefix>_peaks.txt` file is also saved in the output directory, containing the sample index and time (ms) of every detected peak. This is useful for building arrival-time distributions.

© 2025 All rights reserved

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

4.4 fit — Iterative multi-Gaussian decomposition

```
python stratos.py fit \
    --pulse-dir <dir> \
    [--samp-rate 0.025] \
    [--chi2-threshold 1.5] \
    [--max-gaussians 10] \
    [--sn-threshold 3.0] \
    [--output-dir <dir>] \
    [--plot-pdf diagnostics.pdf]
```

The core scientific step of the pipeline. For each pulse file found in `--pulse-dir`, the algorithm iteratively adds Gaussian components until the reduced χ^2 statistic drops below `--chi2-threshold`. The FWHM intercepts of the outermost Gaussians define the pulse envelope.

Flag	Default	Description
`--pulse-dir`	*required*	Directory of pulse txt files from extract step
`--samp-rate`	`0.025`	Time resolution in ms per sample
`--chi2-threshold`	`1.5`	Reduced χ^2 convergence target
`--max-gaussians`	`10`	Hard cap on number of Gaussian component
`--sn-threshold`	`3.0`	Min S/N for a peak to seed a new Gaussian
`--output-dir`	`(same as pulse-dir)`	Where to write `*_fit.txt` files
`--plot-pdf`	*(none)*	Path for diagnostic PDF; omit to skip

Output format: `*_fit.txt`

```
# Multi-Gaussian fit summary for pulse_P0023
# chi2_red          = 1.21
# n_gaussians       = 3
# envelope_left     = 187.3142  (samples)
# envelope_right    = 213.8800  (samples)
# envelope_width_ms = 0.6638
# amp       mu          sigma
  12.452300 197.221400 2.134500
   8.910200 203.441200 1.872300
   5.340100 210.100800 1.540200
```

Choosing `--chi2-threshold`

Value	Behaviour	Recommended for
$\chi^2 = 1.0$	Statistically ideal; may over-fit noisy pulses	High-SNR pulses
$\chi^2 = 1.5$	Good balance of accuracy and speed (default)	Most observations
$\chi^2 = 2.5$	Fewer components; may under-resolve complex pulses	Quick reconnaissance
$\chi^2 = 5.0$	Very loose; single-Gaussian in most cases	Simple profiles

Note

If a pulse cannot be fitted (no significant peaks, or `curve_fit` diverges), the fit file will contain only header comment lines and NaN values will appear in the final properties table.

© 2025 All rights reserved

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

4.5 `fluence` — Radiometric fluence & width measurement

```
python stratos.py fluence \
    --fit-dir <dir> \
    [--samp-rate 0.025] \
    [--sefd 17] [--bw 400e6] [--n-pol 2] \
    [-o pulse_properties.txt]
```

Reads each `*_fit.txt` file alongside its companion pulse data file. Integrates the S/N within the fitted envelope using the radiometer equation to derive fluence in Jy s. The pulse width in ms is derived from the envelope span.

Flag	Default	Description
`--fit-dir`	*required*	Directory with `*_fit.txt` and pulse files
`--samp-rate`	`0.025`	Time resolution in ms per sample
`--sefd`	`17`	System equivalent flux density in Jy
`--bw`	`400e6`	Bandwidth in Hz
`--n-pol`	`2`	Number of summed polarisations
`-o`	`pulse_properties.txt`	Output summary filename

The Radiometer Equation

The flux density of a single time sample is:

$$S_i = \frac{\mathrm{SEFD} \times \mathrm{SNR}_i}{\sqrt{n_{\mathrm{pol}} \times \Delta\nu \times \delta t}} \tag{1}$$

Fluence is the time-integral over the envelope window:

$$\mathcal{F} = \sum_i S_i\,\delta t = \frac{\mathrm{SEFD}\;\delta t}{\sqrt{n_{\mathrm{pol}}\,\Delta\nu\,\delta t}} \sum_i \mathrm{SNR}_i \qquad [\mathrm{Jy\,s}] \tag{2}$$

Output format

```
# pulse_id       width_ms   fluence_jy_s   n_gaussians   chi2_red
pulse_P0001      12.3500    0.00042810      3             1.32
pulse_P0002       8.7250    0.00021550      2             1.14
pulse_P0003       3.1000    0.00008330      1             0.98
```


© 2025 All rights reserved

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

4.6 energy — Convert fluence to spectral energy

```
python stratos.py energy \
    --properties pulse_properties.txt \
    [--distance 3e21] \
    [--bw 250e6] \
    [-o pulse_energies.txt]
```

Converts fluence (Jy s) into spectral energy (ergs) assuming isotropic emission at a known distance D:

$$E = \mathcal{F} \times \Delta\nu \times 4\pi D^2 \times 10^{-23} \qquad [\text{erg}] \tag{3}$$

Flag	Default	Description
`--properties`	*required*	`pulse_properties.txt` from fluence step
`--distance`	`3e21`	Source distance in cm (3×10^{21} cm ≈ 1 kpc)
`--bw`	`250e6`	Bandwidth in Hz
`-o`	`pulse_energies.txt`	Output filename

! Warning

The energy calculation assumes **isotropic emission** and a flat spectral response over the observing band. For pulsars with steep spectral indices this will systematically bias the inferred energy.

© 2025 All rights reserved

Overview p.3
Installation p.4
Quick Start p.5
Command Reference p.6
read p.6
clean p.7
extract p.8
fit p.9
fluence p.10
energy p.11
run p.12
Tutorials p.??
B0355+54 p.??
FRB analysis p.??
Batch mode p.??
Output Files p.??
Parameter Ref. p.??
Algorithm p.??
FAQ p.??

4.7 `run` — Execute the full pipeline end-to-end

```
python stratos.py run \
    --fil <file.fil>  --dm <DM>  --block <N> \
    [--bad-channels c1 c2 ...]  [--samp-rate 0.025] \
    [--distance 160]  [--prominence 8]  [--half-width 200] \
    [--prefix pulse] \
    [--chi2-threshold 1.5]  [--max-gaussians 10]  [--sn-threshold
    3.0] \
    [--sefd 17]  [--bw 400e6]  [--n-pol 2] \
    [--output-dir ./stratos_output]
```

The `run` command chains all steps in the correct order: **read** → **clean** → **extract** → **fit** → **fluence**. It accepts the union of all per-step arguments and creates a self-contained output directory.

• Tip

After a full run you can re-run only the fit and fluence steps with different parameters without re-reading the filterbank — simply call `stratos.py fit` and `stratos.py fluence` pointing at the existing `pulses/` directory.

© 2025 All rights reserved